Name _____

ME435 / CSSE435 Exam 3

Problem	Score
1. MATLAB App Designer *a. Layout*	_____ / 10
1. MATLAB App Designer *b. Functional requirements*	_____ / 30
2. Python Pi *a. Gpiozero*	_____ / 20
2. Python Pi *a. MQTT*	_____ / 20
3. Demo	_____ / 20
Total	_____ / 100

In this part of the exam you can use your computer, links from the course website (slides, labs, etc), Moodle for code submission, MATLAB help, any notes you've made, solutions to your course material, etc. Communication with others during the exam time is prohibited. Using generative AI tools, like Copilot IS allowed (this is very unusual for a Rose exam, but fine on this one). Do try to use AI Wisdom as always and use AI like you would a TA in the course or asking your instructor. AI is to help you get unstuck but not to just do it all for you. You can use AI as/if you get stuck.

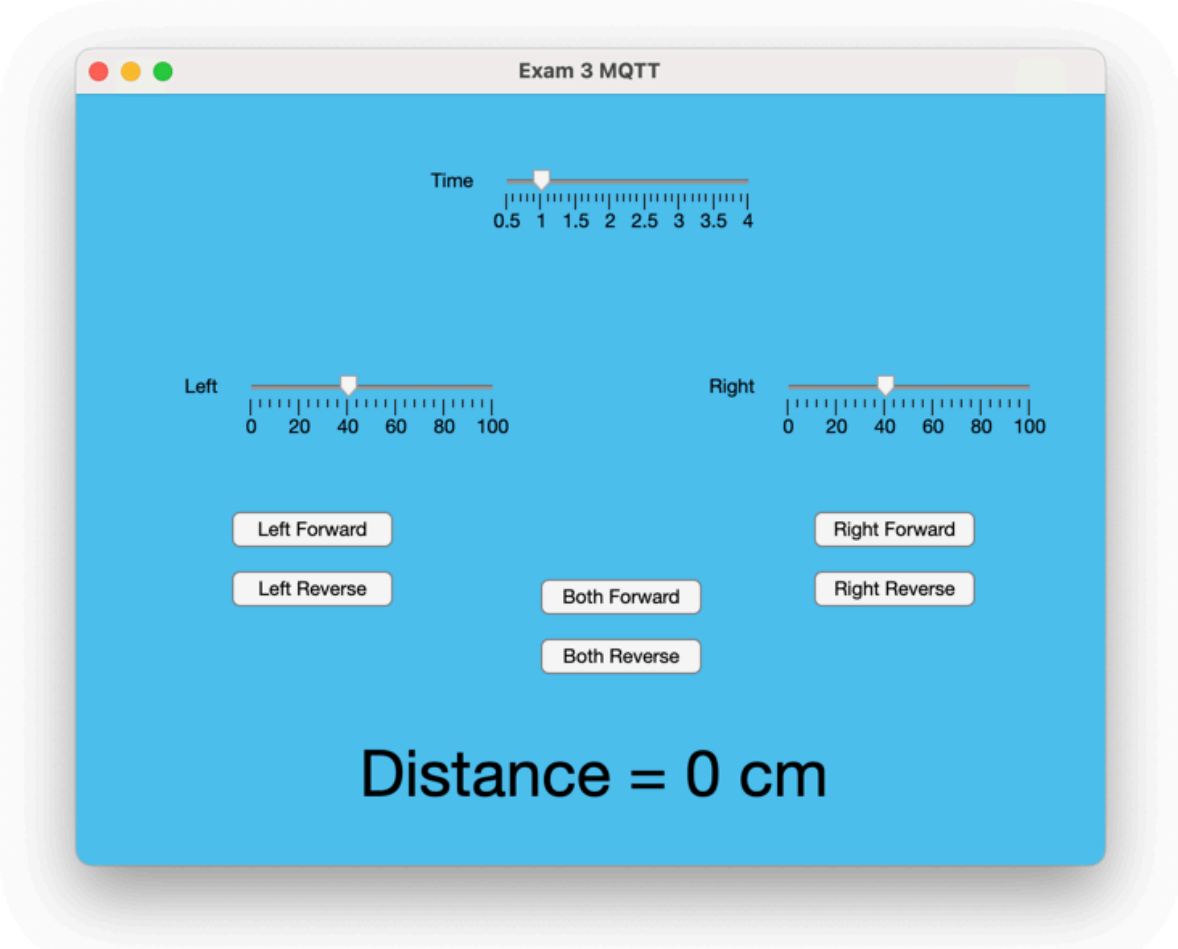
Note, on exams I regularly have you do a few things that are slightly beyond what we have done before. My goal is to see if you can repeat the exact things we've done AND apply your foundation of knowledge to a "slightly" new challenge. So don't be shocked if you are forced to learn small additional skills during the exam.

MATLAB App Designer

This problem is broken up into parts to make partial credit grading easier, but you can (and should) work the entire problem together. You'll be making **a tool for left/right wheel control.** This is not ideal for mecanum, but it will show that you have the skills. Throughout this exam the Left Front and Left Rear motors will always do the exact same thing. Also the Right Front and Right Rear motors will always do the exact same thing. This exam is really designed to be a demo that runs on the table with the car sitting on a box or something so that the wheels aren't on the ground.

Layout - Create a GUI that has 3 sliders (see below), 6 pushbuttons, and a label. Your version can look a bit different, but there are some requirements that you do need to match:

- the figure name says "Exam 3 MQTT" not the default (find and change that property)
- The Time slider has a value of 1 with a range of 0.5 to 4
- The Left and Right sliders have an initial value of 40 with a range of 0 to 100
- Buttons have the labels as show below with a rough layout like shown below
- The label just has a placeholder word for now. Mine says "Distance = 0 cm". It is very wide to allow more text if needed and centered in the GUI.



You can make the background color whatever you like for fun.

Functional requirements - MATLAB Send

Each button will send an MQTT command to your Pi (using a broker and topic names similar to lectures) using a very specific type + payload schema:

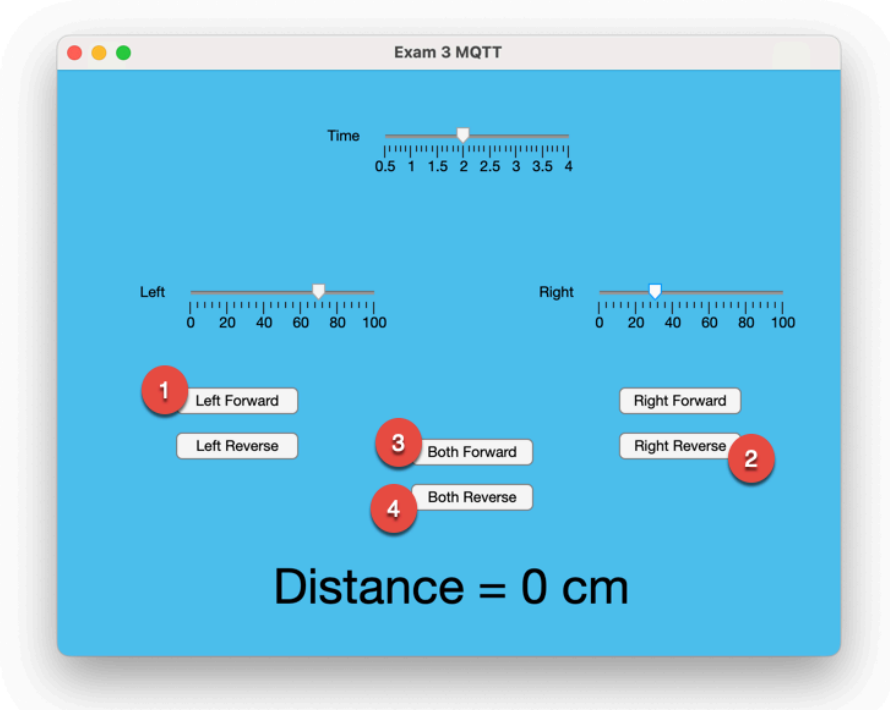
Warning: This is a terrible message schema! It will make it harder than what we did in class. I just want to see that you can do something different. This is well… different!

Button	Type	Payload	Example Payload
Left Forward	**left**	[forward/reverse, time, pwm]	[1, 2.5, 40]
Left Reverse	**left**	[forward/reverse, time, pwm]	[0, 2.5, 40]
Right Forward	**right**	[forward/reverse, time, pwm]	[1, 2.5, 40]
Right Reverse	**right**	[forward/reverse, time, pwm]	[0, 2.5, 40]
Both Forward	**both**	[forward/reverse, time, pwm, pwm]	[1, 2.5, 40, 40]
Both Reverse	**both**	[forward/reverse, time, pwm, pwm]	[0, 2.5, 40, 40]

This is a rather arbitrary choice and there are many solutions, but this is the format you **must** use. The payload means: [forward/reverse, time_slider_value, appropriate_pwm_slider_value]
Then for the "both" command there are **two** PWM values, the **left sent first** then the right second.

Examples:

Let's say I moved the sliders and set them to: time 2 (or near 2), left 70 (or near 70), right 30
Then I hit the buttons (1 at a time) in the order below:



Then MATLAB would send the messages:

- `{"type": "left", "payload", [1, 2.0, 70]}`
- `{"type": "right", "payload", [0, 2.0, 30]}`
- `{"type": "both", "payload", [1, 2.0, 70, 30]}`
- `{"type": "both", "payload", [0, 2.0, 70, 30]}`

Repeat:

Warning: This is a terrible message schema! It will make it harder than what we did in class. I just want to see that you can do something different. This is well… different!

Functional Requirement - Tank Receive

When the Pi receives these command it prints the message using a print statement of the format:

```
Side: Left  Direction: 1   Time: 2.0    Speed: 70
Side: Right  Direction: 0   Time: 2.0    Speed: 30
Side: Both  Direction: 1   Time: 2.0    Speed Left: 70  Speed Right: 30
Side: Both  Direction: 0   Time: 2.0    Speed Left: 70  Speed Right: 30
```

Then it actually performs that command with the motors for the length of time specified.

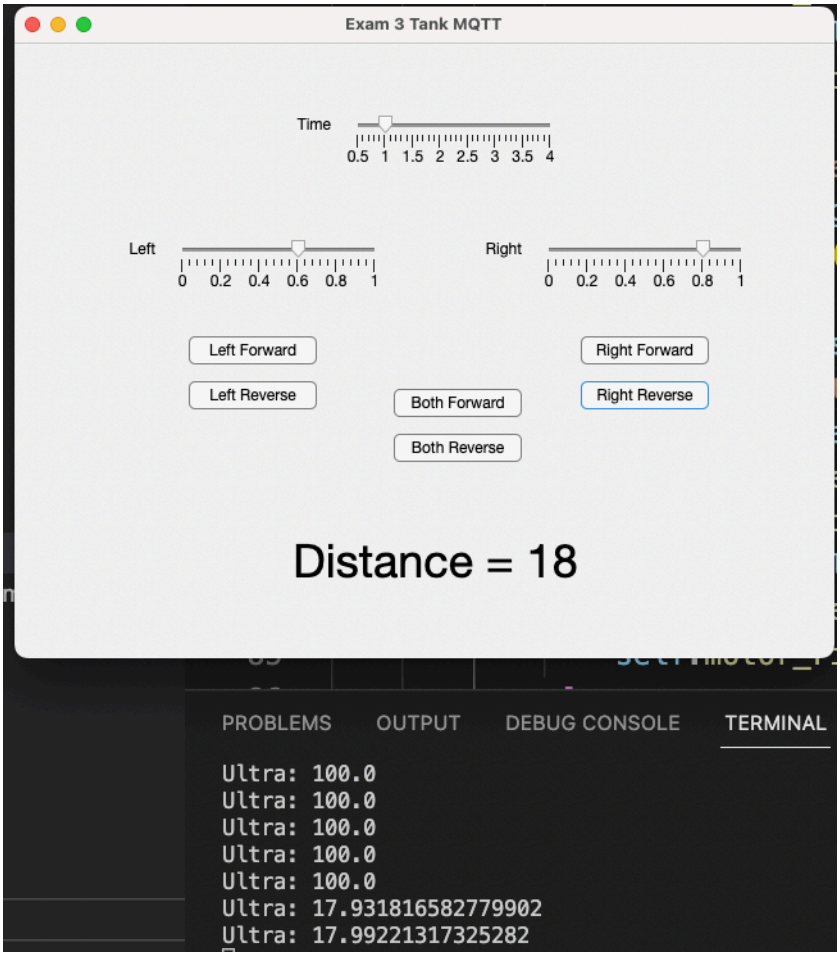
Functional Requirement - Pi Send

Additionally the Pi needs to send the ultrasonic sensor reading **every 2 seconds** using the format:

- `{"type": "ultra", "payload", 26.7}`
- `{"type": "ultra", "payload", 27.1}`
- `{"type": "ultra", "payload", 100.0}`

In addition to sending an MQTT message the python code should also display the reading of the ultrasonic sensor once per 2 seconds just before/after the value is sent via MQTT.

A comment about rounding: You can send as many decimals as you like in the MQTT message but on the MATLAB GUI display only an **integer** in centimeters. For example here you can see that my MQTT message has like 10 decimal points in the reading (see the black area below showing my Pi's logs), but my GUI is just showing an **integer** (the gray MATLAB area).



(older screenshot from the Tank days, but you get the idea)

You will do a live demo of your Exam 3 in class and **submit your MATLAB file and Python files on Moodle.** Usually you can just .zip the files needed and submit 1 .zip file.